

Multi-Label Toxic Comment Classification with Fine-Tuned BERT

An end-to-end deep learning pipeline for detecting toxic, obscene, threatening, and hateful language on Wikipedia talk pages — built with BERT, focal loss, and a Streamlit demo app.

AUTHOR

Sameer Batra

DATASET

Kaggle Jigsaw

MODEL

BERT-base-uncased

BEST MEAN AUC

0.9912

BEST MEAN F1

0.5416

Table of Contents

1.	Introduction & Motivation	3
2.	The Problem: What Is Toxic Comment Classification?	3
3.	Dataset Overview	3
4.	Data Cleaning Pipeline	4
5.	Exploratory Data Analysis (EDA)	4
6.	Model Architecture	5
7.	Handling Class Imbalance: Focal Loss	6
8.	Training Configuration & Strategy	7
9.	Evaluation Metrics Explained	7
10.	Results & Analysis	8
11.	Web Application	9
12.	Project File Structure	9
13.	Key Takeaways & Future Work	10
14.	References	10

SECTION 1

Introduction & Motivation

Every day, millions of people post comments on platforms like Wikipedia, Reddit, and YouTube. While most discussions are healthy, a significant portion contains toxic, threatening, or hateful language. Manually reviewing every comment is impossible at scale — this is where machine learning steps in.

This project builds an automated system that can **read a comment and simultaneously decide whether it is toxic, obscene, threatening, insulting, severely toxic, or identity-based hate** — or any combination of these. Think of it as an AI content moderation assistant trained to flag multiple types of harmful language at once.

🔗 What Makes This Challenging?

Most comments are perfectly fine — only a small fraction are toxic. This **class imbalance** (like finding needles in a haystack) means a naive model can score 95% accuracy by just labeling everything "not toxic." This project specifically tackles that problem using a technique called focal loss.

SECTION 2

The Problem: What Is Toxic Comment Classification?

Traditional text classification assigns one label per piece of text (e.g., "spam" or "not spam"). This project uses **multi-label classification** — each comment can belong to multiple categories simultaneously. For example, a comment might be both toxic AND obscene AND insulting all at once.

● TOXIC

General harmful or rude language that is likely to make someone leave a discussion.

● SEVERE_TOXIC

Extreme toxicity — the most vile and aggressive comments that go beyond regular rudeness.

● OBSCENE

Sexually explicit or grossly offensive language and profanity.

● THREAT

Comments that threaten violence or harm against a person or group.

● INSULT

Inflammatory remarks that demean or belittle another individual or group.

● IDENTITY_HATE

Hate speech targeting someone's identity — race, religion, gender, sexual orientation, etc.

SECTION 3

Dataset Overview

The data comes from the **Kaggle Jigsaw Toxic Comment Classification Challenge**, which contains real comments scraped from Wikipedia talk pages — the behind-the-scenes discussion areas where editors debate article changes. Jigsaw (a technology incubator by Google) compiled and labeled this dataset.

159K

TRAINING
COMMENTS

153K

TEST
COMMENTS

6

TOXICITY
LABELS

~10%

TOXIC
COMMENTS

Why Is Imbalance a Big Deal?

Consider the threat label: only about 0.3% of all comments contain actual threats. If a model predicts "no threat" for every single comment, it would be right 99.7% of the time — but completely useless. This is why special

techniques (discussed in Section 7) are needed to make the model actually learn to detect rare categories.

⚠ **Data Split Used in This Project**

The labeled training file was split **85% for training** and **15% for validation**, giving approximately 135,635 training samples and 23,936 validation samples. The split was stratified — meaning the proportion of toxic comments was kept equal in both halves.

SECTION 4

Data Cleaning Pipeline

Wikipedia talk-page comments are notoriously messy. They contain raw HTML tags, wiki-specific formatting codes, user signatures, URLs, IP addresses, and all sorts of encoding artifacts. Feeding this raw text directly to a model would confuse it. The cleaning pipeline (**data_cleaner.py**) strips all of this away in a carefully ordered sequence.

Each cleaning step is handled by a dedicated, single-purpose class — this modular design makes it easy to add, remove, or debug individual steps without affecting the rest.

1 **HTML Stripper**

Removes any raw HTML tags (e.g., `<b>`, `<div>`) using BeautifulSoup, leaving only the readable text.

2 **Wiki Markup Cleaner**

Strips Wikipedia-specific formatting: internal links like `[[Article Name]]`, templates like `{{citation needed}}`, and heading markup.

3 **URL Anonymiser**

Replaces all HTTP/HTTPS URLs and `www.` links with the placeholder token `[URL]`. This prevents the model from memorizing specific websites.

4 **IP Address Anonymiser**

Replaces IPv4 addresses (e.g., `192.168.1.1`) with `[IP]`. IP addresses appear in Wikipedia when anonymous users make edits.

- 5 Username Anonymiser**
Replaces `[[User:Username]]`, bare `User:Username` patterns, and `@mentions` with `[USER]` tokens to prevent the model from learning user-specific patterns.
- 6 Whitespace Normalizer**
Collapses multiple spaces, tabs, and newlines into single spaces and trims leading/trailing whitespace.
- 7 Unicode Normalizer**
Applies NFKC normalization (converts visually similar characters to their standard forms) and removes invisible control characters.
- 8 Empty Text Handler**
Any comment that becomes completely blank after all the above steps is replaced with the token `[EMPTY]`, preventing empty-string errors.

SECTION 5

Exploratory Data Analysis (EDA)

Before training, the **eda.py** module runs a structured analysis of the dataset to understand the data characteristics. This is critical for making informed modeling decisions — for instance, knowing that threat is rare informs the choice of focal loss.

What Was Analyzed?

The EDA module automatically generates **nine visualizations** and prints tabular statistical reports covering:

LABEL DISTRIBUTION

How many comments fall into each of the 6 toxicity categories. Reveals severe class imbalance — most comments are non-toxic.

LABEL CO-OCCURRENCE

A heatmap showing which labels appear together. For example, obscene and insult frequently co-occur.

TEXT LENGTH ANALYSIS

Distribution of comment lengths broken down by label. Toxic

WORD CLOUDS

Visual word frequency summaries per label — showing the most

comments tend to be longer than neutral ones.

common words in toxic vs. non-toxic comments.

LABEL CARDINALITY

How many labels per comment — most toxic comments have 1–2 labels, few have all 6.

NULL / MISSING CHECK

Heatmap checking for missing values across all columns. The dataset has very few nulls.

Model Architecture

What Is BERT? (Plain English)

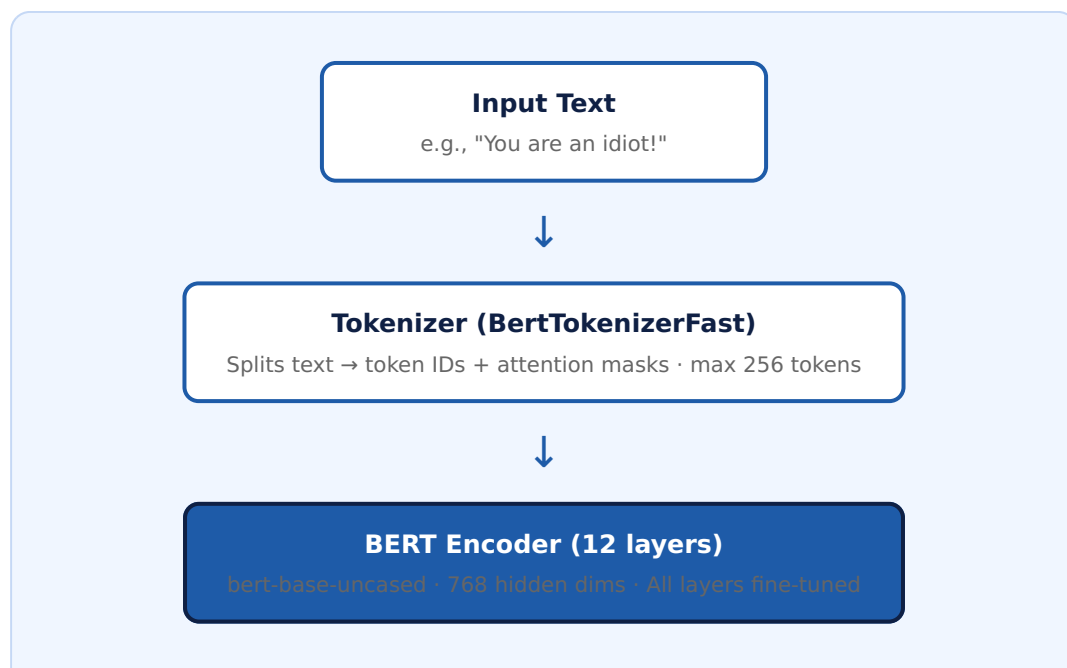
BERT stands for **Bidirectional Encoder Representations from Transformers**. Think of it as a very powerful reading comprehension system that was pre-trained by Google on billions of words from books and websites. Unlike older models that read text left-to-right, BERT reads both directions simultaneously — it understands that the word "bank" means something different in "river bank" vs. "bank account" because it sees all surrounding words at once.

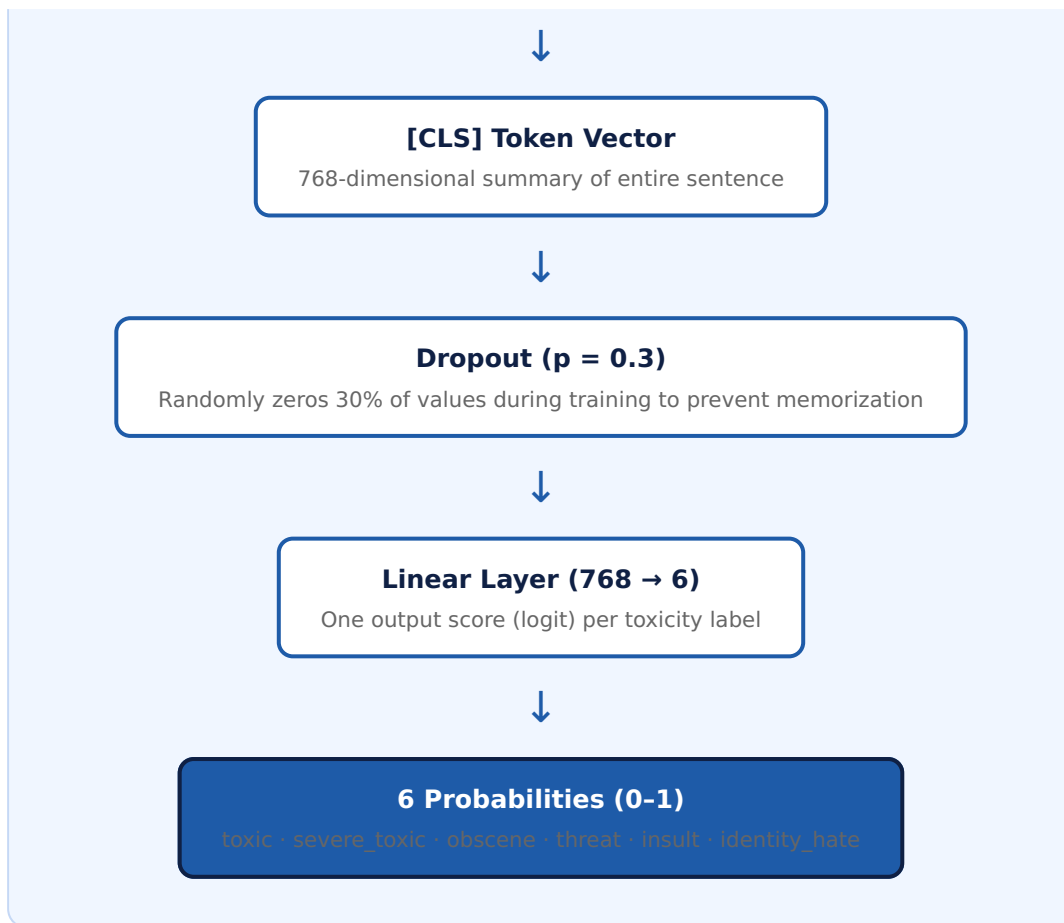
This project uses **bert-base-uncased** — "base" meaning the standard-sized version (not the larger one), and "uncased" meaning all text is lowercased before processing.

🔑 Key BERT Numbers

12 transformer layers · **768-dimensional hidden states** · **110 million parameters** · Trained on BooksCorpus + Wikipedia · Max input length: 512 tokens

Architecture Diagram





Why Use the [CLS] Token?

BERT processes every token in a sentence and outputs a vector for each. The special **[CLS]** token (prepended at the start of every input) is specifically designed to accumulate a whole-sentence summary during training. Using this single 768-dimensional vector as the input to our classifier is a standard and effective approach for sentence-level classification tasks.

Why Dropout?

Dropout is a regularization technique. During training, it randomly "turns off" 30% of the neurons at each step. This forces the network to not rely too heavily on any single feature, which helps it **generalize better to unseen data** rather than just memorizing the training examples.

What Does "Fine-Tuning" Mean?

BERT comes pre-trained — it already understands language from its initial training on billions of words. Fine-tuning means we take that pre-trained model and continue training it on our specific task (toxicity detection) with

a much lower learning rate (**2×10^{-5}**). In this project, all 12 encoder layers are fine-tuned — no layers are frozen — allowing the model to fully adapt its language understanding to toxic content detection.

Handling Class Imbalance: Focal Loss

The hardest technical challenge in this project is the severe **class imbalance**: the vast majority of comments are benign, while rare labels like threat and severe_toxic appear in fewer than 1% of samples. Standard cross-entropy loss treats every example equally — so the model spends most of its learning capacity on easy "not toxic" examples and barely learns rare toxic ones.

Focal Loss (introduced by Lin et al. at Facebook AI in 2017 for object detection) solves this by automatically down-weighting easy examples and focusing learning effort on hard, misclassified ones.

How It Works — Intuitively

STANDARD CROSS-ENTROPY

A comment the model is already 99% confident about "not toxic" still contributes the same loss as a tricky borderline case. Easy examples dominate training.

FOCAL LOSS FIX

Multiplies the loss by $(1 - p_t)^2$. If the model is already 99% confident, this factor is only 0.0001 — nearly zeroing out the gradient. Hard cases get full attention.

The Two Key Components

1. Focal Weight ($\gamma = 2.0$) — The focusing parameter gamma controls how aggressively easy examples are down-weighted. This project uses $\gamma = 2$, a common default:

$$\text{focal_weight} = (1 - p_t)^\gamma \quad \text{where} \quad p_t = p \cdot y + (1-p) \cdot (1-y)$$

p_t is the model's estimated probability for the correct class. Higher p_t = easier example = smaller weight.

2. Class-Balanced Alpha (α_k) — In addition to the focal weight, each label gets a separate balance factor α_k computed directly from training data frequencies:

$$\alpha_k = N_negative / N_total = 1 - positive_rate$$

Rare labels (few positives) get a high α_k , giving their positive examples more weight during training.

✔ **Why Compute Alpha from Training Data Only?**

The class weights (α_k) are calculated from the post-split **training portion only** — never including validation rows. This ensures that validation data cannot "leak" any statistical information into the training objective, keeping evaluation fair and unbiased.

SECTION 8

Training Configuration & Strategy

The training process is orchestrated by **trainer.py** and configured via **train.py**. Multiple techniques are combined to ensure stable, efficient training of the large BERT model.

Setting	Value	Why This Choice?
Optimizer	AdamW	AdamW decouples weight decay from the gradient update, which works better than standard Adam for transformers.
Learning Rate	2×10^{-5}	Standard BERT fine-tuning range — large enough to learn but small enough to not destroy pre-trained weights.
Weight Decay	0.01	L2 regularization to discourage the model from relying on any single large weight value.
Batch Size	32	Balances GPU memory usage with stable gradient estimates.
Epochs	5	Set in train.py (default config allows up to 10, but early stopping prevents overfitting).

Setting	Value	Why This Choice?
Gradient Clipping	max_norm = 1.0	Prevents "exploding gradients" — caps gradient magnitude to keep training numerically stable.
Warmup Ratio	10% of steps	Gradually increases LR from 0 to target during the first 10% of training, preventing early instability.
LR Plateau Decay	×0.5, patience=2	Halves the learning rate if validation metric doesn't improve for 2 epochs.
Early Stopping	patience=5	Stops training if the monitored metric doesn't improve by at least 1e-4 for 5 epochs.
Monitor Metric	mean_f1	Best checkpoint is saved when mean F1 across all labels reaches a new peak.

What Gets Saved?

The best model checkpoint (**best_model.pt**) stores the model weights, optimizer state, the epoch number, the metric value, and the metric name. A full training log (**training_results.csv**) records per-epoch losses and per-label AUC/F1 scores for analysis.

⚠ **README vs. Code Discrepancy:** The README states the model was trained for 3 epochs and uses ROC-AUC as the primary metric. However, the actual code (train.py) runs for **5 epochs** and monitors **mean F1** for checkpointing. The results in this report reflect the actual code behavior.

Evaluation Metrics Explained

Two complementary metrics are used to evaluate the model, each capturing a different dimension of performance. Understanding both is essential to interpreting the results correctly.

ROC-AUC (Area Under the Curve)

ROC-AUC measures how well the model ranks comments — specifically, how often it assigns a higher toxicity probability to an actually toxic comment than to a non-toxic one. A score of **1.0 is perfect; 0.5 is random chance**.

✦ Key Property of AUC

AUC does not depend on any threshold. It evaluates the model purely on whether its probability rankings are correct, making it useful for comparing models and understanding overall discrimination ability — even with imbalanced classes.

F1 Score

F1 is computed by applying a fixed decision threshold of **0.5**: probabilities above 0.5 are predicted as positive, below as negative. It is the harmonic mean of **Precision** (when you predict positive, how often are you right?) and **Recall** (of all actual positives, how many did you catch?):

$$F1 = (2 \times \text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

A score of 1.0 is perfect; 0.0 means the model found nothing useful. $F1 = 0$ for a label if the model predicts zero positives.

Why Does the AUC-F1 Gap Exist?

Notice in the results that AUC is ~ 0.99 while mean F1 is only ~ 0.54 . This is not a contradiction — it reflects a fundamental property of imbalanced classification. The model is excellent at ranking (AUC), but the fixed 0.5 threshold is suboptimal for rare categories. For example, a label with only

0.3% positives might need a threshold of 0.1 to maximize F1 — but the code uses 0.5 for all labels.

 The Fix: Per-Label Thresholds

The most impactful future improvement would be tuning separate probability thresholds for each label on the validation set. This alone could significantly boost mean F1 without changing the model at all.

SECTION 10

Results & Analysis

Training Dynamics Over 5 Epochs

The table below shows how the model improved across each training epoch. The training loss decreases consistently — meaning the model is continuously learning. The validation loss rises slightly after epoch 2, indicating mild overfitting begins around epoch 3. The **best checkpoint was saved at epoch 3**, when mean F1 peaked at 0.5416.

Epoch	Train Loss	Val Loss	Mean ROC-AUC	Mean F1	Status
1	0.0037	0.0023	0.9888	0.4223	Warming Up
2	0.0021	0.0022	0.9909	0.4382	Improving
3 ★	0.0016	0.0033	0.9912	0.5416	Best Checkpoint
4	0.0013	0.0033	0.9911	0.5358	Slight Decline
5	0.0011	0.0035	0.9900	0.5129	Overfitting

Per-Label Performance at Best Epoch (Epoch 3)

The table below shows how well the model performs on each toxicity category individually. AUC is uniformly excellent across all labels (all above 0.988). F1 varies dramatically, which directly reflects how rare each label is in the dataset.

Label	ROC-AUC	F1 Score	Relative Rarity	Notes
toxic	0.9881	0.802	Common (~10%)	Highest F1 — most training examples available
obscene	0.9943	0.766	Moderate (~5%)	Strong performance; closely related to toxic
insult	0.9905	0.731	Moderate (~5%)	Good F1; shares patterns with toxic/obscene
severe_toxic	0.9910	0.381	Rare (~1%)	Low F1 due to sparsity; high AUC shows good ranking
identity_hate	0.9919	0.357	Rare (~1%)	Few examples make threshold-based F1 difficult
threat	0.9912	0.212	Very Rare (~0.3%)	Lowest F1 — fewest examples; hardest to detect

✓ **Key Insight: High AUC + Low F1 = Threshold Problem, Not Model Quality**

The model's near-perfect AUC scores (0.98–0.99) across all labels — including rare ones like threat — confirm that the model genuinely learned to distinguish toxic from non-toxic comments. The low F1 for rare labels is a **calibration problem**, not a fundamental failure. Tuning per-

label thresholds on the validation set would dramatically improve F1 with no model changes.

Web Application

The project includes an interactive demo web application built with **Streamlit** — a Python framework for building data apps with minimal code. The app allows anyone to type in a comment and instantly see toxicity probability scores for all six labels, displayed with color-coded Plotly charts.

How the App Works

1 User types a comment

The app renders a text input field. The user types any comment they want to analyze.

2 ToxicityPredictor runs inference

The `inference.py` module loads the model (`unitary/toxic-bert` by default) from Hugging Face Hub, tokenizes the input, runs it through the model, and applies sigmoid to get probabilities.

3 Results displayed with color coding

Probabilities below 0.25 appear green (safe), 0.25–0.5 yellow (caution), 0.5–0.75 orange (warning), above 0.75 red (high toxicity). A Plotly bar chart visualizes all six scores.

⚠ Important: Two Different Models

The web app uses **unitary/toxic-bert** (a pre-existing Hugging Face model) by default — not the custom-trained `JigsawBERTClassifier`. To use the custom-trained model, the `MODEL_NAME` in `config.py` would need to be updated to point to the saved checkpoint.

Project File Structure

The codebase is organized into single-purpose modules, making each component independently testable and replaceable. Below is a complete map of every file and its role.

File	What It Does	Run It With
<code>eda.py</code>	Loads raw CSVs, computes statistics, generates 9 visualization PNG files in <code>eda_outputs/</code>	<code>python eda.py</code>
<code>data_cleaner.py</code>	Runs the 8-step cleaning pipeline on raw train/test CSVs, outputs <code>train_clean.csv</code> and <code>test_clean.csv</code>	<code>python data_cleaner.py</code>
<code>dataset.py</code>	JigsawDataset class + DataLoaderFactory: handles tokenization, stratified splitting, and PyTorch DataLoader creation	Imported by <code>train.py</code>
<code>model.py</code>	Defines JigsawBERTClassifier: the BERT backbone + dropout + linear head	Imported by <code>train.py</code>
<code>focal_loss.py</code>	AlphaComputer (computes per-label class weights) + FocalLoss module	Imported by <code>train.py</code>
<code>trainer.py</code>	Full training loop: forward pass, backward pass, gradient clipping,	Imported by <code>train.py</code>

File	What It Does	Run It With
	metrics tracking, checkpointing, early stopping, CSV logging	
train.py	Main entry point: wires all configs together, builds data loaders and model, calls Trainer.train()	python train.py
config.py	App-level constants: model name, max length, threshold, label names, display names, color thresholds	Imported by app.py
inference.py	ToxicityPredictor class: loads a Hugging Face model, tokenizes input, returns 6 probability scores	Imported by app.py
app.py	Streamlit web interface: dark theme, Plotly charts, text input, calls ToxicityPredictor	streamlit run app.py
requirements.txt	Full dependency list for training + app	pip install -r requirements.txt
requirements_app.txt	Minimal dependency list for app-only deployment	pip install -r requirements_app.txt

Key Takeaways & Future Work

What This Project Demonstrated

✓ What Worked Well

BERT's pre-training is enormously powerful — even with just 5 epochs and a simple linear head, the model achieves near-perfect AUC (0.99) on all 6 labels. Focal loss successfully prevented the model from ignoring rare labels entirely, and the modular codebase makes each component independently testable.

⚠ Where Improvement Is Needed

The fixed 0.5 threshold hurts F1 on rare labels. The model is actually very good at scoring threats and identity hate — it just needs a lower threshold to "fire" on them. This is the single biggest gap between the model's potential and its measured F1.

Lessons Learned

- **Class imbalance dominates F1 on rare labels** — even when AUC is near-perfect, a global threshold fails rare categories. Always report both metrics when working with imbalanced data.
- **Choice of checkpoint metric matters** — monitoring mean_F1 vs. mean_AUC saves different model weights and optimizes for different downstream behaviors.
- **Wiki-specific cleaning is essential** — the specialized anonymization of URLs, IPs, and usernames prevents the model from learning spurious correlations with specific users or websites.
- **Validation data must be kept strictly separate** — computing α_k from training-only data ensures that the loss function doesn't "peek" at the validation distribution.

Future Improvements (Prioritized)

#	Improvement	Expected Impact
1	Per-label threshold tuning — find optimal threshold per label on validation set	High — would significantly boost F1 for threat, identity_hate, severe_toxic
2	Weighted sampling — oversample rare label examples during DataLoader construction	High — complements focal loss for very rare labels
3	Longer training with LR scheduling — allow more epochs with aggressive plateau decay	Medium — AUC is near ceiling; F1 may improve further
4	Adversarial / robustness testing — test with deliberate misspellings (e.g., "id!ot") that evade detection	Medium — important for production deployment
5	Long-document handling — BERT truncates at 256 tokens; some comments are longer	Low — most comments are short enough

SECTION 14

References

- **Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019).** BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT 2019.
- **Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017).** Focal Loss for Dense Object Detection. Proceedings of ICCV 2017.
- **Loshchilov, I., & Hutter, F. (2019).** Decoupled Weight Decay Regularization. ICLR 2019. (AdamW optimizer)
- **Kaggle.** Jigsaw Toxic Comment Classification Challenge. Dataset available at kaggle.com/c/jigsaw-toxic-comment-classification-challenge.
- **Wolf, T. et al. (2020).** HuggingFace's Transformers: State-of-the-art Natural Language Processing. EMNLP 2020.

- **unitary/toxic-bert.** Pre-trained toxicity detection model available on Hugging Face Hub. huggingface.co/unitary/toxic-bert

Report generated from source code documentation and training_results.csv. All quantitative results are sourced directly from the training run logs. Author: Sameer Batra · NLP Final Project.